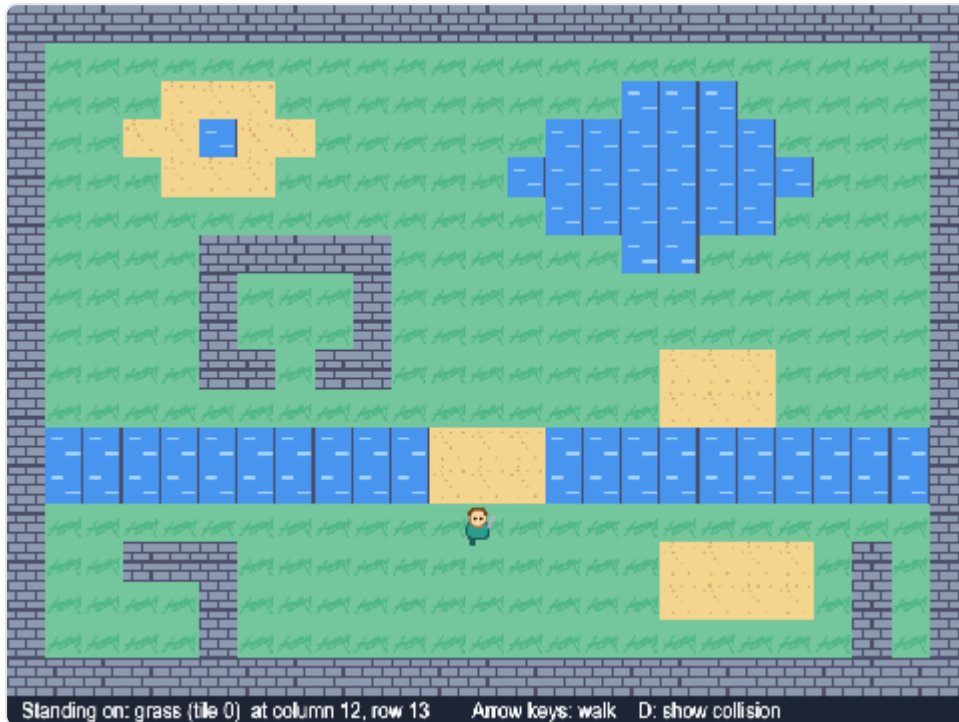


Tilemaps

Most 2D games with a world to walk round - dungeons, islands, platform levels, puzzle boards - are built from **tiles**: small square pictures laid out on a grid. In this chapter you build tile worlds in code: a map written as a 2D array of numbers, a tile painter that changes the map while the game runs, and a random cave far bigger than the screen, with a camera that follows the player and a minimap in the corner.



What you will learn

- why games use tiles, and what a **tileset** and a **tile index** are
- how to turn a 2D array into a map with `this.make.tilemap`, `addTilesetImage` and `createLayer`
- how to make some tiles solid with `setCollision`, and stop an Arcade Physics sprite walking into them
- how to find the tile at a point, and convert between pixels and tile columns and rows
- how to change the map while the game runs, with `putTileAt`, `removeTileAt` and `putTilesAt`, and what an empty tile (`-1`) is
- how to make a world bigger than the screen: camera bounds, `startFollow`, and a second camera as a minimap

- the difference between free movement and grid movement

The projects

Project	What it shows
<code>ch10_array_map</code>	a top-down map from a 2D array; a player who cannot walk into walls or water
<code>ch10_tilePainter</code>	click to paint tiles; number keys choose the tile; save and load in <code>localStorage</code> ; print the map as code
<code>ch10_scrolling_world</code>	a 60 x 40 random cave made in code, two layers, a camera that follows the player, and a minimap

Chapter 11 does the same things with maps drawn in [Tiled](#), a free map editor. Everything in this chapter still applies there - Tiled just writes the numbers for you.

Why tiles?

Imagine drawing a level as one big picture. A world 60 tiles by 40 is 1920 x 1280 pixels - about 10 MB of graphics memory for one level, and every change means repainting it. Worse, the game has no idea what is *in* the picture: which pixels are wall, which are water.

Tiles fix all three problems:

- **memory:** the pictures are drawn once, in a small **tileset** image, and reused as often as you like. `simple_tiles.png` , which the first two projects use, is 128 x 32 pixels - four tiles
- **level design:** a level is just a grid of numbers. Change a number, and the level changes. A new level is a new grid, not a new picture
- **collision and rules:** the game knows exactly what is at every place, because it knows the number there. "Is this a wall?" is a question about a number, not about pixels

A tileset is one picture; a map is a grid of numbers that pick from it

simple_tiles.png - 128 x 32 pixels, four 32 x 32 tiles



0

grass



1

water



2

wall



3

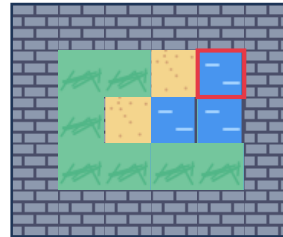
sand

each tile's INDEX is its place in the picture, counting from 0 - left to right, then row by row in a taller tileset

-1 means "no tile": nothing is drawn

```
const LEVEL: number[][] = [  
  [2, 2, 2, 2, 2, 2],  
  [2, 0, 0, 3, 1, 2],  
  [2, 0, 3, 1, 1, 2],  
  [2, 0, 0, 0, 0, 2],  
  [2, 2, 2, 2, 2, 2],  
];
```

make.tilemap



LEVEL[1][4] is 1:
row 1, column 4
is water

rows first: LEVEL[row][column] - the array looks like the map

Each tile in a tileset has an **index**: its place in the picture, counting from 0, left to right and then row by row. The asset library has three tilesets, all with 32 x 32 tiles (the full lists are in `assets/README.md`):

Tileset	Tiles	Good for
<code>simple_tiles.png</code>	4: 0 grass, 1 water, 2 wall, 3 sand	a first map; this chapter's first two projects
<code>dungeon_tiles.png</code>	16: floors, walls, doors, stairs, water, a void...	top-down dungeons; <code>ch10_scrolling_world</code>
<code>platform_tiles.png</code>	16: grass tops, dirt, bricks, spikes, ladders...	side-on platform games (Chapters 11 and 15)

A map from an array

The level is data

`ch10_array_map` keeps its map in a file of its own:

`src/level.ts`

```
export const LEVEL: number[][] = [
  [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
  [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 0, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 3, 3, 1, 3, 3, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0],
  [2, 0, 0, 3, 3, 1, 3, 3, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0],
  ...
  [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
];

// where the player starts: a column and a row, counted in TILES from the
// top left (0, 0)
export const START_COLUMN = 7;
export const START_ROW = 7;
```

`number[][]` is an array of arrays of numbers - Java's `int[][]`. There is one inner array for each **row**, top to bottom, and one number for each **column**, left to right. Because it is written row by row, the code looks like the map: you can see the walls round the edge (2), the pond with its sandy beach (3 round a 1), and the lake. The map is 25 columns by 18 rows; at 32 pixels a tile that is 800 x 576 pixels, leaving a 24-pixel strip at the bottom of the game for a line of text.

What the numbers mean lives in another small file:

`src/tiles.ts`

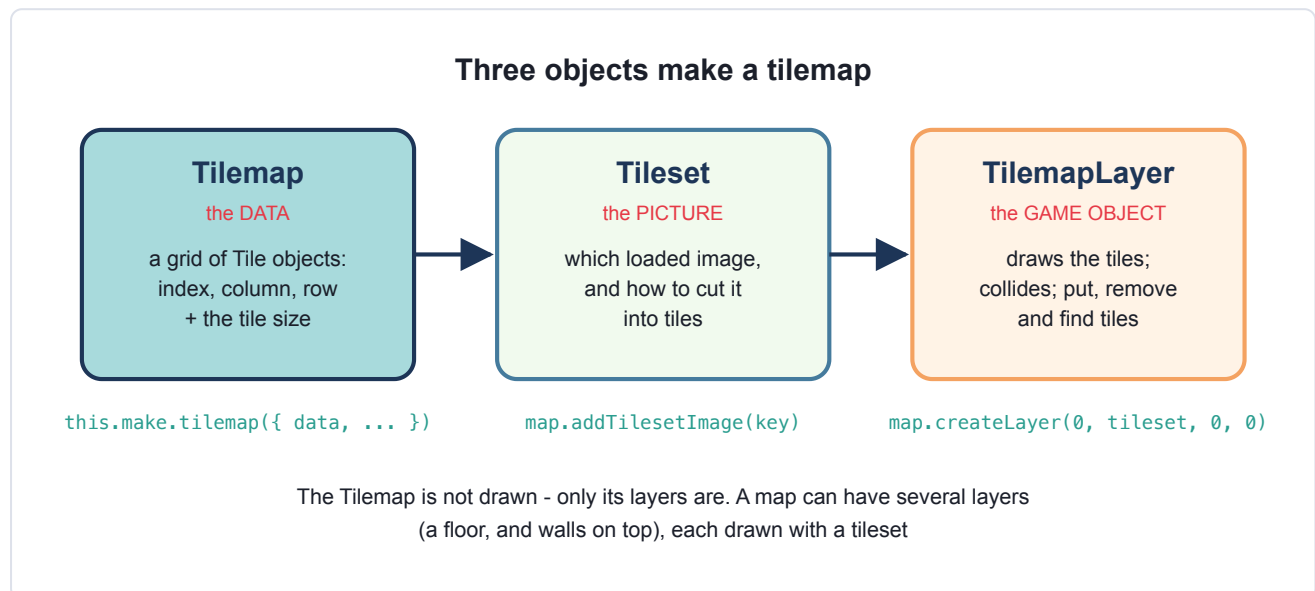
```
export const TILES_KEY = "tiles";
export const TILES_FILE = "assets/tilesets/simple_tiles.png";

export const TILE_SIZE = 32;

export const GRASS = 0;
export const WATER = 1;
export const WALL = 2;
export const SAND = 3;
```

Note - rows first. `LEVEL[row][column]` - that is, `LEVEL[y][x]` - is the tile at a place. Almost everyone writes `LEVEL[x][y]` once. With a square map the mistake can hide for a while; with this 25 x 18 map it gives `undefined` as soon as `x` passes 17.

Three objects make a tilemap



A tileset is loaded like any other picture - Phaser cuts it into tiles later:

`src/scenes/GameScene.ts`

```
preload(): void {
    // a tileset is loaded as a plain image - Phaser cuts it into tiles later
    this.load.image(TILES_KEY, TILES_FILE);
    this.load.spritesheet(HERO_KEY, HERO_FILE, { frameWidth: HERO_FRAME_SIZE,
    frameHeight: HERO_FRAME_SIZE });
}
```

Then `create()` builds the map in five steps:

```

create(): void {
    // 1. the map: the data, and the size of one tile in pixels
    const map = this.make.tilemap({ data: LEVEL, tileWidth: TILE_SIZE,
    tileHeight: TILE_SIZE });

    // 2. the tilesheet: which loaded image the tile indexes refer to
    const tilesheet = map.addTilessetImage(TILES_KEY)!;

    // 3. the layer: the game object that draws the tiles. A map made from an
    array has one layer, 0.
    // createLayer can make a TilemapLayer or (if asked) a GPU-only
    TilemapGPULayer, so its return
    // type is "either"; we did not ask for a GPU layer, so we tell
    TypeScript which one it is
    this.layer = map.createLayer(0, tilesheet, 0, 0) as
    Phaser.Tilemaps.TilemapLayer;

    // 4. collision: tiles with these indexes block physics bodies
    this.layer.setCollision([WATER, WALL]);

    // the player starts in the middle of a tile. tileToWorldXY gives the
    tile's TOP LEFT corner
    const start = map.tileToWorldXY(START_COLUMN, START_ROW)!;
    this.player = new Player(this, start.x + TILE_SIZE / 2, start.y +
    TILE_SIZE / 2);

    // 5. the player bumps into the colliding tiles
    this.physics.add.collider(this.player, this.layer);
    ...
}

```

1. `this.make.tilemap(...)` makes a `Tilemap`: the **data**. It turns every number in the array into a `Tile` object that knows its index, its column and its row. A `Tilemap` is *not* drawn - it is a model, not a view. (`this.add.tilemap(...)` makes one too, taking its settings as separate arguments instead of an object; a `Tilemap` is never on the display list, whichever you use)
2. `map.addTilessetImage(name, key)` makes a `Tilesheet`: which loaded image the numbers refer to, and how to cut it up. The first argument is the tilesheet's *name* and the second is the image's key; leave the key out and the name is used as the key, as here. (In Chapter 11 the name matters: it must match the name the tilesheet has inside the Tiled map)

3. `map.createLayer(0, tileset, x, y)` makes a `TilemapLayer` : a **game object** that draws the tiles, at `(x, y)` . A map from an array has exactly one layer, number 0
4. `layer.setCollision([...])` marks every tile with those indexes as solid
5. `this.physics.add.collider(player, layer)` - the same collider as Chapter 8, with a tilemap layer on one side. Arcade Physics now separates the player from every solid tile it touches

The `!` after `addTilesetImage(...)` and `tileToWorldXY(...)` is TypeScript's **non-null assertion**. Both methods say they might return `null` (the tileset's image might not be loaded; the map might have no layer) - `!` says "not this time". If it is wrong, the error comes at the next line that uses the value, so keep it for things you know are there.

The `as Phaser.Tilemaps.TilemapLayer` is needed because `createLayer` can also make a `TilemapGPULayer` - a faster, WebGL-only layer, new in Phaser 4, made when you pass `true` as a fifth argument. The return type says "one or the other"; we know which, and say so. (The GPU layer is for enormous maps; it does not show `putTileAt` changes until you call `generateLayerDataTexture()` , so this chapter uses the ordinary one.)

One more setting, in `src/main.ts` : `pixelArt: true` . Tile art is small and blocky, and the game is scaled to fit the window; this tells Phaser to scale it without smoothing, so the tiles stay crisp.

The player

`src/objects/Player.ts` is an Arcade Physics sprite (Chapter 8) using `topdown_hero.png` , with a walk animation for each direction (Chapter 7). It does not move itself: it sets its **velocity** from the arrow keys, and the physics world moves it - which is what lets the collider stop it.

`src/objects/Player.ts`

```
// normalise, so that walking diagonally is not faster than walking straight
const velocity = new Phaser.Math.Vector2(dx, dy).normalize().scale(SPEED);
this.setVelocity(velocity.x, velocity.y);
```

`dx` and `dy` are -1, 0 or 1. Holding RIGHT and DOWN together gives `(1, 1)` , which is about 1.41 long - so without `normalize()` (which makes it length 1) the player would go 41% faster diagonally.

One more line matters for tiles:

```
// a body smaller than the picture, centred on it
this.setSize(BODY_SIZE, BODY_SIZE);
```

The picture is 32 x 32 - exactly one tile. A body that size would have to be lined up perfectly, to the pixel, to fit through the house's one-tile door; one pixel out and it catches on the wall. A 20 x 20 body slips through easily, and still looks right. Making bodies a little smaller than their tiles is normal practice in tile games.

Seeing what collides

Press **D** in the game to see which tiles are solid, and which of their edges Phaser checks:



src/scenes/GameScene.ts

```
// renderDebug draws every tile that collides, and its colliding edges
this.layer.renderDebug(this.debugGraphics, {
  tileColor: null, // tiles that do
  not collide: nothing
  collidingTileColor: new Phaser.Display.Color(230, 57, 70, 120),
  faceColor: new Phaser.Display.Color(255, 255, 255, 255),
});
```

The white lines are the **interesting faces**: only the edges of a solid tile that face a non-solid tile. The edge between two wall tiles can never be touched, so Phaser does not check it.

That is one reason tile collision is fast, and it is why a player sliding along a straight wall does not snag at the joins between tiles.

Which tile am I on?

The strip under the map says what the player is standing on, every frame:

src/scenes/GameScene.ts

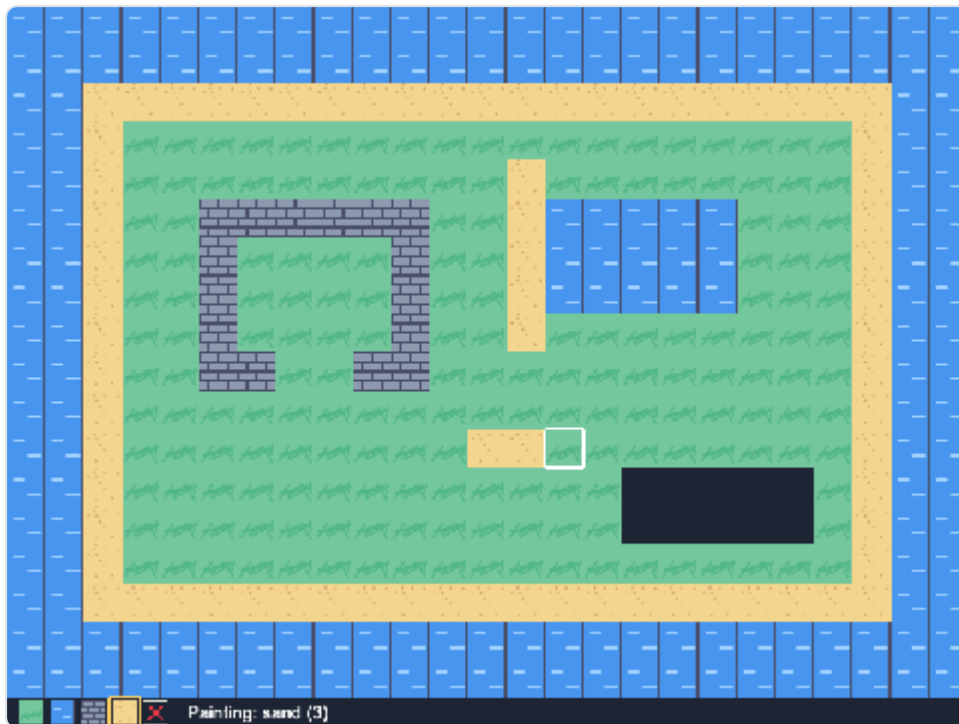
```
override update(): void {
    // which tile is under the middle of the player? getTileAtWorldXY takes
    // PIXELS...
    const tile = this.layer.getTileAtWorldXY(this.player.x, this.player.y);

    // ...and the tile knows its own column and row (x and y, counted in
    // tiles) and index
    if (tile) {
        const name = TILE_NAMES[tile.index];
        this.infoText.setText(
            `Standing on: ${name} (tile ${tile.index}) at column ${tile.x}, row
            ${tile.y}` +
            `      Arrow keys: walk      D: show collision`,
        );
    }
}
```

`getTileAtWorldXY(x, y)` takes a position in **pixels** and gives back the `Tile` there - or `null` if there is no tile (off the map, or an empty cell). Watch the names: a `Tile`'s `x` and `y` are its column and row, **in tiles**. Its position in pixels is `pixelX` and `pixelY`. A `Tile` has plenty more - `collides`, `properties`, `tint`, `alpha` - but `index`, `x` and `y` are the ones you use most.

`if (tile)` is JavaScript's short way of saying `if (tile !== null)` - `null` counts as false.

Changing the map while the game runs



A tilemap is not fixed once it is made. `ch10_tilePainter` is a small level editor: choose a tile with the number keys (1-4, or 0 for an eraser) or the palette under the map, then click or drag.

An empty map

This time there is no array. The map is made empty, at the right size, and filled with grass:

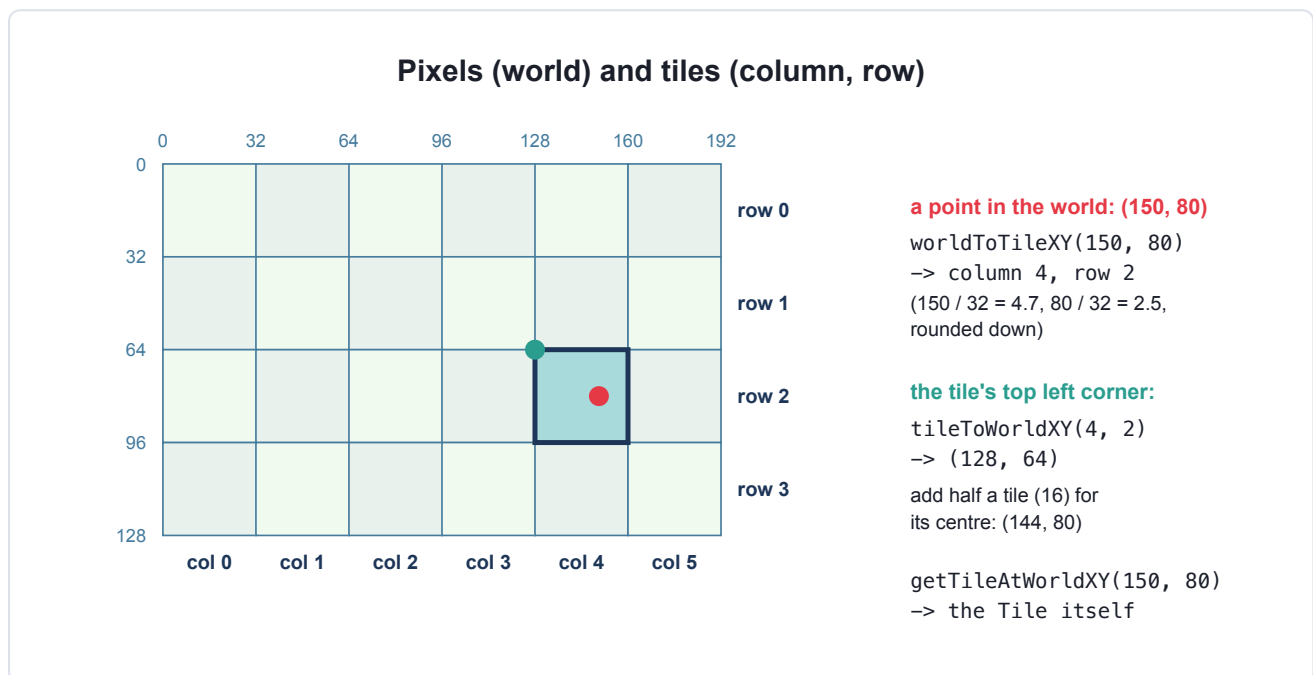
`src/scenes/PainterScene.ts`

```
// no data this time: an empty map of the right size, and a blank layer to
// paint on
const map = this.make.tilemap({
  tileWidth: TILE_SIZE,
  tileHeight: TILE_SIZE,
  width: MAP_COLUMNS,
  height: MAP_ROWS,
});
const tileset = map.addTilesetImage(TILES_KEY!);
this.layer = map.createBlankLayer("ground", tileset!);
this.layer.fill(GRASS);
```

`createBlankLayer(name, tileset)` makes a layer the size of the map, full of empty tiles. A map made this way can have as many layers as you like, each with its own name - `ch10_scrolling_world` has two. `fill(index)` sets every tile in the layer (or, with more arguments, a rectangle of it).

Pixels and tiles

A click happens at a position in pixels; the map works in columns and rows. Two methods convert:



`src/scenes/PainterScene.ts`

```
// which tile is under the pointer? worldToTileXY turns pixels into a column
// and a row
private tileUnder(pointer: Phaser.Input.Pointer): Phaser.Math.Vector2 | null
{
  const cell = this.layer.worldToTileXY(pointer.worldX, pointer.worldY);
  const onMap = cell.x >= 0 && cell.x < MAP_COLUMNS && cell.y >= 0 && cell.y
  < MAP_ROWS;
  return onMap ? cell : null;
}
```

- `worldToTileXY(x, y)` - pixels to tiles. It divides by the tile size and rounds **down**, so every pixel from 128 to 159 across is column 4. It gives a `Vector2` whose `x` is the column and `y` the row. It happily gives columns and rows outside the map (a click on the palette is row 18), so the code checks

- `tileToWorldXY(column, row)` - tiles to pixels: the tile's **top left corner**. The white frame that follows the mouse uses it:

```
const corner = this.layer.tileToWorldXY(cell.x, cell.y);
this.highlight.setPosition(corner.x, corner.y);
```

Why `pointer.worldX` rather than `pointer.x`? `x` is where the pointer is on the **screen**; `worldX` is where that is in the **world**, allowing for the camera. Here the camera never moves, so they are the same - but in a scrolling world they are not, and `worldX` is the one that matches the map. Get into the habit now.

Putting and removing tiles

`src/scenes/PainterScene.ts`

```
private paint(pointer: Phaser.Input.Pointer): void {
  const cell = this.tileUnder(pointer);
  if (cell === null) {
    return; // below the map - the palette handles its own clicks
  }

  if (this.current === EMPTY) {
    this.layer.removeTileAt(cell.x, cell.y); // leaves a hole:
    nothing is drawn there
  } else {
    this.layer.putTileAt(this.current, cell.x, cell.y);
  }
}
```

- `putTileAt(index, column, row)` changes one tile. The change shows on the next frame
- `removeTileAt(column, row)` empties it. Nothing is drawn there, so you see the game's background colour through the hole

An empty cell has index `-1` - "no tile". It is not the same as 0, which is a real tile (grass, here). In an array you write `-1` for an empty cell, and `ch10_scrolling_world` uses that to put one layer on top of another.

Painting is triggered by two events:

```
// paint when the button goes down, and while it is held and the mouse moves
this.input.on("pointerdown", (pointer: Phaser.Input.Pointer) =>
this.paint(pointer));
this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
  this.hover(pointer);
  if (pointer.isDown) {
    this.paint(pointer);
  }
});
```

"pointermove" fires whenever the pointer moves, button or no button; `pointer.isDown` says whether a button is held - so dragging paints a line.

Note - `setCollision` remembers the **indexes** you gave it, not the tiles that had them. A wall tile put down later with `putTileAt` is solid straight away; a wall removed is solid no more. The painter has no player to bump into things, but the same is true in any game: change the map, and the collisions change with it.

Reading the map back, saving and loading

To save the map, the painter turns the layer back into a 2D array - the same shape as `level.ts`:

`src/scenes/PainterScene.ts`

```
private toArray(): number[][] {
  const rows: number[][] = [];
  for (let row = 0; row < MAP_ROWS; row++) {
    const indexes: number[] = [];
    for (let column = 0; column < MAP_COLUMNS; column++) {
      const tile = this.layer.getTileAt(column, row); // null where
there is no tile
      indexes.push(tile === null ? EMPTY : tile.index);
    }
    rows.push(indexes);
  }
  return rows;
}
```

`getTileAt(column, row)` is `getTileAtWorldXY`'s twin, taking a column and row instead of pixels. Both give `null` for an empty cell - so the code writes `-1` there.

Saving and loading use the browser's `localStorage`, as the high-score table did in Chapter 6. S saves; L loads:

```
private saveMap(): void {
    localStorage.setItem(SAVE_KEY, JSON.stringify(this.toArray()));
    this.showMessage("Saved");
}
```

```
// what comes back is only text - check it really is a map of the right size
before using it.
// JSON.parse throws an exception if the text is not JSON at all
let data: unknown = null;
try {
    data = JSON.parse(saved);
} catch {
    // leave data as null - isMap says no
}
if (!this.isMap(data)) {
    this.showMessage("The saved map is damaged");
    return;
}

// putTilesAt puts a whole 2D array down at once, starting at column 0, row
0. -1 becomes empty
this.layer.putTilesAt(data, 0, 0);
```

`putTilesAt(array, column, row)` puts a whole 2D array of indexes down in one go, with its top left corner at that column and row. It is how you would stamp a ready-made room, or a tree two tiles tall, into a map.

`unknown` is TypeScript's type for "could be anything - check before you use it". It is the honest type for something read back from storage, which anyone can edit in the browser's developer tools. The check is a method with an unusual return type:

```
// true if value is MAP_ROWS arrays of MAP_COLUMNS tile indexes (or -1)
private isMap(value: unknown): value is number[][] {
    return Array.isArray(value) && value.length === MAP_ROWS &&
        value.every((row) =>
            Array.isArray(row) && row.length === MAP_COLUMNS &&
            row.every((index) => Number.isInteger(index) && index >= EMPTY &&
index < TILE_COUNT)
        );
}
```

`value is number[][]` is a **type guard**: a method that returns a boolean, and also tells the compiler "if this returns true, `value` is a `number[][]`". After `if (!this.isMap(data)) return;` the compiler knows `data` is a map, and lets it be passed to `putTilesAt` with no `as`. Java has nothing quite like it; the nearest is `instanceof` with a pattern (`if (o instanceof String s)`), which only works for classes. `every(test)` is true if the test is true for every element.

Printing a level for your code

Press **P** and the painter writes the map to the browser's console (F12, or Cmd+Option+I on a Mac, then the Console tab) as TypeScript:

```
// write the map to the browser's console as TypeScript, ready to paste into
level.ts
private print(): void {
    const lines = this.toArray().map((row) => ` ${row.join(", ")}],`);
    console.log(`export const LEVEL: number[][] =
[\n${lines.join("\n")}\n];`);
    this.showMessage("Printed to the browser's console");
}
```

The painter's map is the same size as `ch10_array_map`'s, with the same tiles - so you can paint a level, print it, and paste it over `LEVEL` in that project's `level.ts`. (Make sure the tile at `START_COLUMN`, `START_ROW` is one the player can stand on.) A tool that writes code for you is a game developer's oldest trick.

The palette - passing a function

The palette is a class of its own. It knows nothing about maps; when a button is clicked, it calls a function it was given:

src/objects/Palette.ts

```
constructor(scene: Phaser.Scene, x: number, y: number, onChoose: (index: number) => void) {  
  ...  
  button.on("pointerdown", () => onChoose(index));  
}
```

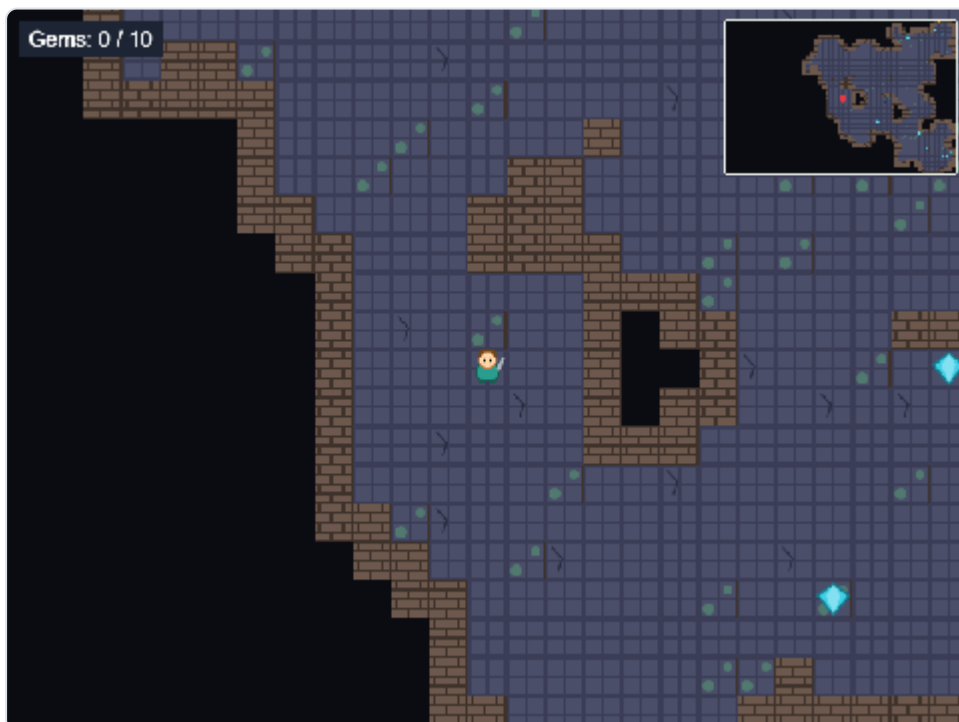
src/scenes/PainterScene.ts

```
this.palette = new Palette(this, 20, STRIP_Y, (index) =>  
  this.choose(index));
```

`(index: number) => void` is a **function type**: "a function that takes a number and returns nothing". In Java you would declare the parameter as an `IntConsumer` and pass a lambda; in TypeScript functions are ordinary values, and their types are written out like this.

The palette's buttons are small `Image`s, one per tile. To show one tile as an `Image`, the painter loads `simple_tiles.png` a **second** time, under another key, with `load.spritesheet` - so each tile is also a frame.

A world bigger than the screen



`ch10_scrolling_world` is a cave 60 tiles by 40 - 1920 x 1280 pixels, more than five screens - made in code, and different every time. Walk round it and find the ten gems; R makes a new cave.

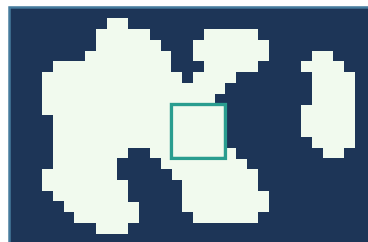
A cave made in code

`src/Cave.ts` makes the cave. It is a plain class, not a game object and not a scene: it knows nothing about Phaser, tiles or pictures, only which cells are rock and which are floor.

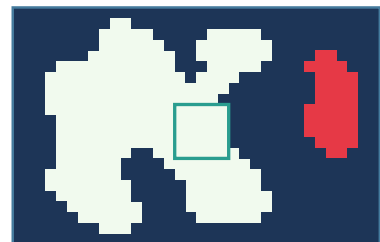
Making a cave: noise, smoothed, then cut-off caves filled in



1. random rock (47%)



2. smoothed 5 times



3. unreachable floor filled

dark = rock, light = floor, green = the cleared start, red = floor the player could never reach, now rock

It uses a well-known recipe called **cellular automata**:

1. fill the grid with random rock - a little under half of it (random "noise")
2. **smooth** it, five times: each cell becomes rock if 5 or more of the 9 cells around it (itself included) are rock. Lone rocks disappear, and clumps grow into walls. What was noise becomes smooth, natural-looking caves
3. clear a space for the player to start in; then **flood fill** from there - visit every floor cell that can be reached, one neighbour at a time - and turn any floor that was *not* reached into rock. Otherwise a gem could be put in a cave the player can never get to

Each step is a short method in `Cave.ts` - `randomRock()`, `smooth()`, `clearStart()` and `fillUnreachable()` - built from nothing more than nested `for` loops over rows and columns. The one subtle point is in `smooth()`: it builds a **new** grid, so that every cell is judged on the old one, not on cells already changed in this pass.

Keeping the cave (the **model**) apart from the tilemap (the **view**) is worth the extra class. The scene decides what the cave *looks* like; the cave decides what it *is*. You could draw the same cave with `platform_tiles.png`, or as coloured squares in a minimap, without touching `Cave.ts`.

Two layers

The scene draws the cave with two layers, one on top of the other:

src/scenes/WorldScene.ts

```
// layer 1, the floor: every cell, mostly plain, some cracked or mossy.
// weightedRandomize picks a
// random index for each tile - a weight of 12 is 12 times as likely as a
// weight of 1
const floor = map.createBlankLayer("floor", tileset!);
floor.weightedRandomize([
  { index: FLOOR, weight: 12 },
  { index: FLOOR_CRACKED, weight: 1 },
  { index: FLOOR_MOSSY, weight: 1 },
]);

// layer 2, the walls: drawn on top of the floor, and empty (-1) where the
// cave is open
const walls = map.createBlankLayer("walls", tileset!);
walls.putTilesAt(this.wallTiles(cave), 0, 0);
walls.setCollisionByExclusion([EMPTY]); // every tile that is not
// empty collides
```

The floor layer is floor everywhere - even under the rock, where it cannot be seen. The wall layer is **empty** (-1) wherever the cave is open, so the floor shows through. Layers are drawn in the order they are made, like any game objects, so walls are on top.

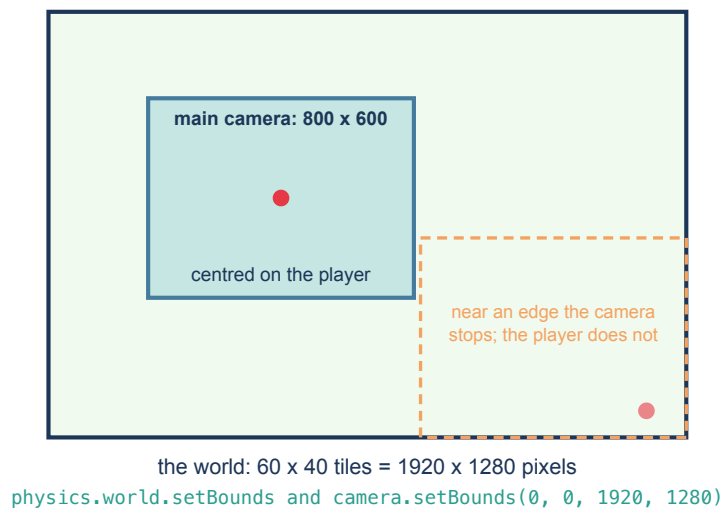
Only the wall layer collides. `setCollisionByExclusion([-1])` means "every index *except* these": whatever wall tiles the scene chose, they are all solid - with no list to keep up to date.

`wallTiles()` turns the cave into indexes. It looks at each rock cell's **neighbours** to choose its picture: bricks where the rock is next to the floor, black "void" deep inside the rock, and now and then a torch on a wall with floor just below it. Choosing a tile from its neighbours is called **auto-tiling**; bigger tilesets have a picture for every combination of neighbours.

The camera

A camera is a window onto the world. Until now the main camera has always looked at the rectangle from (0, 0) to (800, 600), which was the whole game. Now the world is bigger:

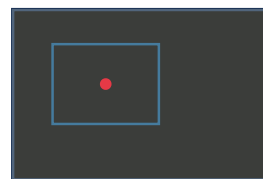
A world bigger than the screen



`startFollow(player, true)`
the camera's `scrollX` / `scrollY` change every frame to keep the player centred

`setBounds(...)`
never shows anything outside the map

`hud.setScrollFactor(0)`
stays in the same place on screen



a second camera,
zoom 0.1: the
whole world in
192 x 128 pixels

`cameras.add(x, y, 192, 128)`

`src/scenes/WorldScene.ts`

```
// the world is bigger than the screen. The camera follows the player, but
// never shows
// anything outside the map; the physics world is the size of the map too
const width = map.widthInPixels;
const height = map.heightInPixels;
this.physics.world.setBounds(0, 0, width, height);
this.cameras.main.setBounds(0, 0, width, height);
this.cameras.main.startFollow(this.player, true); // true: round to
whole pixels - no shimmer
```

- `startFollow(target)` moves the camera every frame to keep the target in the middle of the screen. It changes the camera's `scrollX` and `scrollY` - the world position of the screen's top left corner. (Extra arguments, `lerpX` and `lerpY` between 0 and 1, make it catch up gently instead of sticking exactly)
- `setBounds(x, y, width, height)` stops the camera at the edges of the map. Without it, walking to the edge would show empty space beyond
- `this.physics.world.setBounds(...)` : the physics world is 800 x 600 unless you say otherwise. Here the walls keep the player in anyway, but anything with `setCollideWorldBounds(true)` would otherwise bounce off an invisible wall in the middle of the map

The HUD must not scroll with the world:

```
this.hudText.setScrollFactor(0);           // stays put on the screen while the
camera moves
```

A **scroll factor** of 1 (the default) moves an object fully with the camera; 0 not at all - it is fixed to the screen; 0.5 moves it half as far, which is how parallax backgrounds are made.

A minimap is just another camera

A scene can have several cameras, each drawing the same world into its own rectangle of the screen. The minimap is a second camera in the top right corner, zoomed out to a tenth:

```
const minimap = this.cameras.add(MINI_X, MINI_Y, MINI_WIDTH, MINI_HEIGHT);
minimap.setZoom(MINI_ZOOM);
minimap.setBounds(0, 0, width, height);
minimap.centerOn(width / 2, height / 2);
minimap.setBackgroundColor(0x000000);

// a big red dot on the player: only the minimap shows it
this.marker = this.add.circle(this.player.x, this.player.y, MARKER_RADIUS,
0xe63946);
this.cameras.main.ignore(this.marker);
```

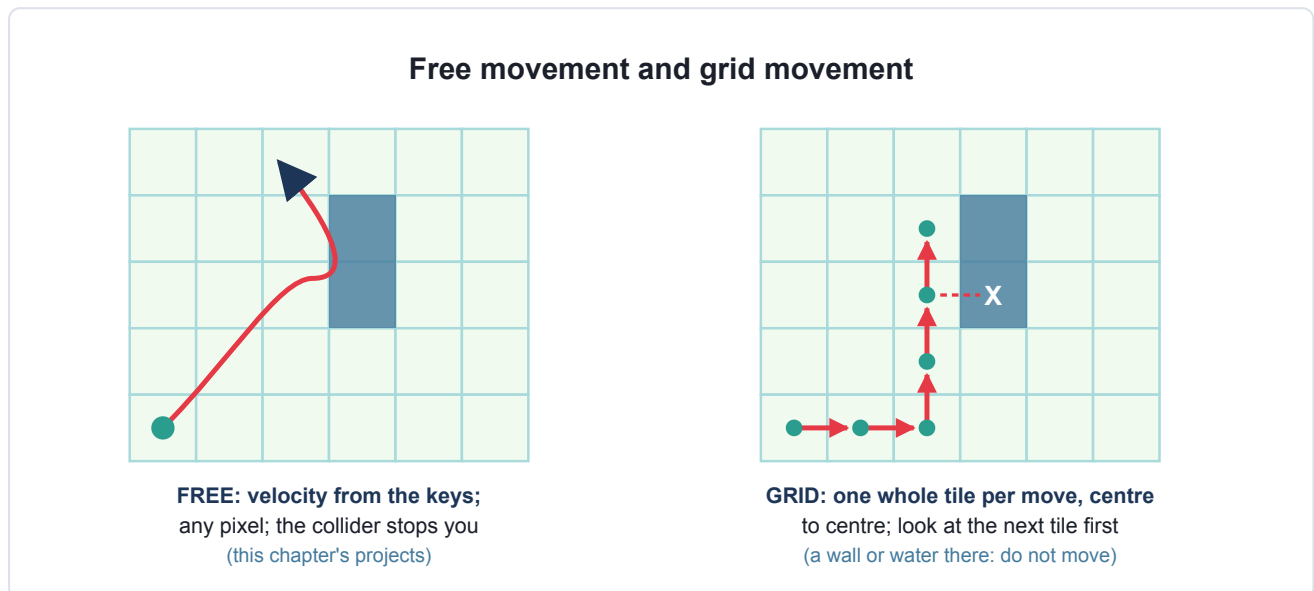
At zoom 0.1, the 1920 x 1280 world fits exactly into 192 x 128 pixels. Every camera draws every game object - unless told to `ignore` it. The player, at a tenth of its size, would be a 3-pixel speck on the minimap, so a big red circle follows the player (in `update()`) and the **main** camera ignores it: it is seen only on the minimap. The other way round, the minimap ignores the HUD text and its own frame, which belong to the main camera's screen.

Starting again

R restarts the scene, and `create()` makes a new `Cave`. Chapter 2's lesson applies: the scene object is reused, so `init()` resets the gem count. And the player is the **same** `Player` **class** as in `ch10_array_map`, unchanged - it never knew how big the world was, or that a camera was following it.

Free movement and grid movement

All three projects use **free movement**: the player can be at any pixel, moves smoothly, and the physics collider stops it at walls. Many tile games - Pokémon, Sokoban, most roguelikes - use **grid movement** instead: the player is always on a tile, and each move goes one whole tile.



Grid movement needs no physics at all. The player's position is really a column and a row. To move, you work out the **next** tile, look at it (`getTileAt`) and decide - is it a wall, water, a door?

- and only then move. The move itself is usually a short **tween** (Chapter 7) from one tile's centre to the next, with input ignored until it finishes, so the player glides rather than teleports.

Which to choose? Free movement suits action games, where fine positioning and speed matter. Grid movement suits puzzles and turn-based games, where the rules are about tiles: "boxes can be pushed one square", "each monster moves one square after you". It is also much simpler to get exactly right - there are no half-overlaps with walls to worry about.

Common mistakes

Note - the map is invisible, but the player still bumps into invisible walls: the key given to `addTilesetImage` does not match a loaded image, so it returns `null` and the layer has no pictures. The browser's console says `Texture key "tilez" not found`. The walls still work because collision is about indexes, not pictures. Keep keys in constants.

Note - the player walks straight through walls: either `setCollision` was not called (or was given the wrong indexes), or there is no `this.physics.add.collider(player, layer)`. You need both. Press D in `ch10_array_map` to see which tiles collide.

Note - the player stands in the corner of a tile, not the middle: `tileToWorldXY` gives the tile's **top left** corner. Add half a tile.

Note - do not name a scene method `load`, `add`, `make`, `physics`, `cameras`, `time` or `input`. They are already fields of every scene, and TypeScript says so, in several confusing ways at once - for a `load()` method, one of them is `Property 'load' in type 'PainterScene' is not assignable to the same property in base type 'Scene'`. That is why the painter's methods are `saveMap()` and `loadMap()`.

Summary

- a **tileset** is one picture of many equal tiles; each tile has an **index**, counting from 0. A map is a grid of indexes; `-1` means an empty cell
- `this.make.tilemap({ data, tileWidth, tileHeight })` makes the map's data from a 2D array, written row by row: `data[row][column]`
- `map.addTilesetImage(key)` says which image to use; `map.createLayer(0, tileset, x, y)` makes the layer that draws it; `map.createBlankLayer(name, tileset)` makes an empty layer for a map made in code

- `layer.setCollision([indexes])` (or `setCollisionByExclusion`) plus `this.physics.add.collider(sprite, layer)` makes tiles solid
- `getTileAtWorldXY` and `getTileAt` find a tile; `worldToTileXY` and `tileToWorldXY` convert between pixels and tiles (the tile's top left corner)
- `putTileAt`, `removeTileAt`, `putTilesAt`, `fill` and `weightedRandomize` change the map at any time
- for a world bigger than the screen: `camera.setBounds`, `camera.startFollow`, `physics.world.setBounds`, and `setScrollFactor(0)` for anything fixed to the screen
- a second camera (`this.cameras.add`, `setZoom`, `ignore`) makes a minimap
- keep generated levels as plain data, separate from the tilemap that draws them

Challenges

1. **A new island** (*ch10_array_map*) - Change the map: add a second bridge across the river, a maze of walls in the bottom half, and a moat of water round the house (with a sand path to the door). Make the player start somewhere else. You only need to change `level.ts` - or paint the map in `ch10_tilePainter` and paste it in.
2. **Sand slows you down** (*ch10_array_map*) - Make the player walk at half speed on sand. The `Player` class should not have to know about tiles: give it a way to be told how fast to go, and let the scene tell it.
3. **Eyedropper** (*ch10_tilePainter*) - Right-clicking a tile on the map should *choose* that tile (as if its number key had been pressed) instead of painting. Right-clicking an empty cell chooses the eraser. The browser's own right-click menu should not appear over the game.
4. **Treasure tiles** (*ch10_scrolling_world*) - Replace the gem sprites with **chest tiles** (`dungeon_tiles.png` index 8) put into the map itself. Walking into a chest collects it: it disappears from the map, the coin sound plays, and the HUD counts it. *Hint:* the chests could go on the wall layer, where they would stop the player - look up `setTileIndexCallback`, or the callback a collider can take. Or put them on a third layer and use `overlap`.
5. **A map from strings** (*ch10_array_map*) - Numbers are hard to read. Write the level as an array of strings instead - one string per row, one character per tile: `#` wall, `~` water, `.` grass, `s` sand, and `@` for where the player starts (on grass). Turn it into the `number[]`

[] the tilemap needs, and find the start position from the @. *Hint:* a `Record<string, number>` can map characters to indexes; `row.split("")` turns a string into an array of characters. Check that every row is the same length.

6. **Grid movement** (*ch10_array_map*) - Change the player to move one whole tile at a time: tap an arrow key to move one tile; hold it to keep moving. The player glides from tile centre to tile centre, cannot move onto walls or water, and cannot change direction halfway between tiles. *Hint:* physics is no longer needed. Keep the player's column and row; before moving, look at the next tile with `getTileAt`; move with a tween (Chapter 7) and ignore the keys until it completes. `tileToWorldXY` gives the corner of the tile to move to.

Previous: [Chapter 9 - 2D physics](#) · Next: [Chapter 11 - Tilemaps with Tiled](#)